SAVITRIBAI PHULE PUNE UNIVERSITY

A MINI PROJECT REPORT ON

Manual Testing of an E-Commerce Website Using Test Cases and Bug Reports

SUBMITTED TOWARDS THE
PARTIAL FULFILLMENT OF THE REQUIREMENTS OF
LABORATORY PRACTICE IV (SOFTWARE TESTING AND QUALITY ASSURANCE)

BACHELOR OF ENGINEERING (Computer Engineering)

BY

Vishwajeet Tulse	Exam No: _____
Student Name 2	Exam No: _____
Student Name 3	Exam No: _____
Student Name 4	Exam No: _____

Under The Guidance of

Prof. Guide Name

Sinhgad Institutes

DEPARTMENT OF COMPUTER ENGINEERING
NBN SINHGAD SCHOOL OF ENGINEERING
10/1, AMBEGAON (BK), PUNE-411041

2026-27

Sinhgad Institutes

NBN SINHGAD SCHOOL OF ENGINEERING
DEPARTMENT OF COMPUTER ENGINEERING

CERTIFICATE

This is to certify that the Mini Project Entitled

Manual Testing of an E-Commerce Website

Submitted by

Vishwajeet Tulse	Exam No: _____
Student Name 2	Exam No: _____
Student Name 3	Exam No: _____
Student Name 4	Exam No: _____

is a bonafide work carried out by Students under the supervision of Prof. Guide Name and it is submitted towards the partial fulfillment of the requirement of Bachelor of Engineering (Computer Engineering) for the course Laboratory Practice IV.

Prof. Guide Name
Internal Guide
Dept. of Computer Engg.

Dr. S. P. Bendale
H.O.D
Dept. of Computer Engg.

Dr. S. P. Patil
Principal
NBN Sinhgad School of Engineering

Signature of Internal Examiner

Signature of External Examiner

PROJECT APPROVAL SHEET

Manual Testing of an E-Commerce Website Using Test Cases and Bug Reports

Is successfully completed by

Vishwajeet Tulse	Exam No: _____
Student Name 2	Exam No: _____
Student Name 3	Exam No: _____
Student Name 4	Exam No: _____

at

DEPARTMENT OF COMPUTER ENGINEERING

NBN SINHGAD SCHOOL OF ENGINEERING, PUNE

SAVITRIBAI PHULE PUNE UNIVERSITY, PUNE

ACADEMIC YEAR 2026-2027

Prof. Guide Name	Dr. S. P. Bendale
Internal Guide	H.O.D
Dept. of Computer Engg.	Dept. of Computer Engg.

Abstract

Manual testing is the foundation of software quality assurance: a tester studies the requirements, designs test cases, executes them by hand in the real application and records what actually happens. This mini project applies the complete manual testing process to **Swag Labs** (`https://www.saucedemo.com`), a publicly available demo e-commerce website that offers login, a product catalogue, a shopping cart and a checkout flow. The site provides six test user accounts, several of which deliberately expose functional, visual and performance defects, which makes it an ideal subject for learning defect detection and reporting.

The work followed the standard test life cycle. A test plan defined the scope, strategy, environment and entry/exit criteria. Twenty test scenarios were derived from the application's features and expanded into thirty-nine test cases written in an Excel sheet with pre-conditions, steps, test data, expected and actual results, status and priority. Black-box techniques (equivalence partitioning, boundary values, error guessing) were used to design positive and negative cases, in particular for the login and checkout forms. Every test case was executed manually in Microsoft Edge on Windows 11 with the appropriate test user, and evidence screenshots were captured.

Twenty-eight test cases passed and eleven failed. The failures were analysed and consolidated into nine defect reports, each with severity, priority, steps to reproduce, expected versus actual behaviour and a screenshot. The defects range from a critical checkout blocker (the Last Name field cannot be filled by the problem user) to wrong product images, broken sorting, wrong prices, missing input validation on the checkout form, a cart that survives logout, and a seven-second login delay. A requirement traceability matrix links every requirement to its test cases, and a summary sheet gives module-wise results. The outcome is a complete, reproducible manual testing package that demonstrates test planning, test design, execution, defect reporting and traceability as taught in the Software Testing and Quality Assurance course.

Acknowledgments

It gives us great pleasure in presenting the mini project report on ***'Manual Testing of an E-Commerce Website'****.*

We would like to take this opportunity to thank our internal guide ***Prof. Guide Name*** *for giving us all the help and guidance we needed. We are really grateful to them for their kind support. Their valuable suggestions were very helpful.*

We are also grateful to ***Dr. S. P. Bendale****, Head of Computer Engineering Department, NBN Sinhgad School of Engineering, for his indispensable support and suggestions.*

In the end our special thanks to the ***laboratory staff of the Department of Computer Engineering*** *for providing various resources such as the laboratory with all needed software platforms and continuous Internet connection for our project.*

Vishwajeet Tulse
Student Name 2
Student Name 3
Student Name 4
(B.E. Computer Engg.)

Contents

1 Synopsis **1**
- 1.1 Project Title . . . 2
- 1.2 Project Option . . . 2
- 1.3 Internal Guide . . . 2
- 1.4 Sponsorship and External Guide . . . 2
- 1.5 Technical Keywords (As per ACM Keywords) . . . 2
- 1.6 Problem Statement . . . 2
- 1.7 Abstract . . . 2
- 1.8 Goals and Objectives . . . 3
- 1.9 Relevant mathematics associated with the Project . . . 3
- 1.10 Names of Conferences / Journals where papers can be published . . . 4
- 1.11 Review of Conference/Journal Papers supporting Project idea . . . 4
- 1.12 Plan of Project Execution . . . 4

2 Technical Keywords **5**
- 2.1 Area of Project . . . 6
- 2.2 Technical Keywords . . . 6

3 Introduction **7**
- 3.1 Project Idea . . . 8
- 3.2 Motivation of the Project . . . 8
- 3.3 Literature Survey . . . 8

4 Problem Definition and scope **10**
- 4.1 Problem Statement . . . 11
  - 4.1.1 Goals and objectives . . . 11
  - 4.1.2 Statement of scope . . . 11
- 4.2 Major Constraints . . . 11
- 4.3 Methodologies of Problem solving and efficiency issues . . . 11
- 4.4 Outcome . . . 12
- 4.5 Applications . . . 12
- 4.6 Hardware Resources Required . . . 12
- 4.7 Software Resources Required . . . 12

5 Project Plan **13**
- 5.1 Project Estimates . . . 14
  - 5.1.1 Reconciled Estimates . . . 14
  - 5.1.2 Project Resources . . . 14
- 5.2 Risk Management w.r.t. NP Hard analysis . . . 14

5.2.1 Risk Identification . . . 14
5.2.2 Risk Analysis . . . 17
5.2.3 Overview of Risk Mitigation, Monitoring, Management . . . 17
5.3 Project Schedule . . . 17
5.3.1 Project task set . . . 17
5.3.2 Task network . . . 17
5.3.3 Timeline Chart . . . 17
5.4 Team Organization . . . 17
5.4.1 Team structure . . . 17
5.4.2 Management reporting and communication . . . 18

6 Software requirement specification 19
6.1 Introduction . . . 20
6.1.1 Purpose and Scope of Document . . . 20
6.1.2 Overview of responsibilities of Developer . . . 20
6.2 Usage Scenario . . . 20
6.2.1 User profiles . . . 20
6.2.2 Use-cases . . . 21
6.2.3 Use Case View . . . 22
6.3 Data Model and Description . . . 22
6.3.1 Data Description . . . 22
6.3.2 Data objects and Relationships . . . 23
6.4 Functional Model and Description . . . 23
6.4.1 Data Flow Diagram . . . 23
6.4.2 Activity Diagram: . . . 24
6.4.3 Non Functional Requirements: . . . 24
6.4.4 State Diagram: . . . 25
6.4.5 Design Constraints . . . 25
6.4.6 Software Interface Description . . . 25

7 Detailed Design Document using Appendix A and B 26
7.1 Introduction . . . 27
7.2 Architectural Design . . . 27
7.3 Data design (using Appendices A and B) . . . 27
7.3.1 Internal software data structure . . . 27
7.3.2 Global data structure . . . 27
7.3.3 Temporary data structure . . . 27
7.3.4 Database description . . . 28
7.4 Component Design . . . 28
7.4.1 Class Diagram . . . 28

8 Project Implementation 29
8.1 Introduction . . . 30
8.2 Tools and Technologies Used . . . 30
8.3 Methodologies/Algorithm Details . . . 30
8.3.1 Algorithm 1/Pseudo Code . . . 30
8.3.2 Algorithm 2/Pseudo Code . . . 31
8.4 Verification and Validation for Acceptance . . . 31

9 Software Testing **32**
9.1 Type of Testing Used 33
9.2 Test Cases and Test Results 33

10 Results **36**
10.1 Screen shots 37
10.2 Outputs 38

11 Deployment and Maintenance **40**
11.1 Installation and un-installation 41
11.2 User help 41

12 Conclusion and Future Scope **42**

13 References **44**

Annexure A Laboratory assignments on Project Analysis of Algorithmic Design **46**

Annexure B Laboratory assignments on Project Quality and Reliability Testing of Project Design **48**

Annexure C Project Planner **50**

Annexure D Reviewers Comments of Paper Submitted **52**

Annexure E Plagiarism Report **54**

Annexure F Term-II Project Laboratory Assignments **56**

Annexure G Information of Project Group Members **58**

List of Figures

5.1 Project timeline (Gantt chart) 17

6.1 Use case diagram 22
6.2 Entity relationship of test artifacts 23
6.3 Level 0 DFD 23
6.4 Level 1 DFD 23
6.5 Activity diagram of test execution 24
6.6 State transition diagram (defect life cycle) 25

7.1 Architecture of the testing process 27
7.2 Class diagram of test artifacts 28

10.1 Test case sheet (first rows of the Excel workbook) 37
10.2 Defect report sheet 37
10.3 Login test evidence 37
10.4 Checkout happy-path evidence 38
10.5 Defect evidence 38

List of Tables

4.1 Hardware Requirements 12

5.1 Time estimates 14
5.2 Risk Table 15
5.3 Risk Probability definitions [11] 15
5.4 Risk Impact definitions [11] 15

6.1 Use Cases 21

9.1 Module-wise test results (pass percentage 71.8%) 33
9.2 Selected test cases (full list of 39 in the Excel workbook) 34
9.3 Defects logged 35

C.1 Project planner 51

CHAPTER 1
SYNOPSIS

1.1 Project Title

Manual Testing of an E-Commerce Website Using Test Cases and Bug Reports

1.2 Project Option

Internal project (mini project for Laboratory Practice IV, Software Testing and Quality Assurance).

1.3 Internal Guide

Prof. Guide Name

1.4 Sponsorship and External Guide

Not applicable. The project is not sponsored.

1.5 Technical Keywords (As per ACM Keywords)

1. Software and its engineering
   (a) Software verification and validation
      i. Software testing and debugging
      ii. Acceptance testing, functional testing
   (b) Software creation and management
      i. Software defect analysis, requirements analysis
2. Information systems
   (a) World Wide Web, Web applications (electronic commerce)

1.6 Problem Statement

Perform manual testing of an e-commerce website (Swag Labs, `https://www.saucedemo.com`) by preparing a test plan and test scenarios, designing positive and negative test cases in an Excel sheet, executing them by hand with the available test users, recording actual results, and documenting all defects found in structured bug reports with severity, priority, reproduction steps and evidence.

1.7 Abstract

- E-commerce websites must handle login, product listing, cart and checkout correctly; defects in these flows directly cause lost sales and customer distrust.

- Swag Labs is a public demo store that includes deliberately defective test users, so it allows realistic defect hunting without harming a live business.
- A test plan, 20 scenarios and 39 test cases were prepared using black-box techniques.
- All test cases were executed manually in Microsoft Edge; 28 passed and 11 failed.
- Nine defects were reported with severity, priority, steps and screenshots; a traceability matrix and summary complete the package.

1.8 Goals and Objectives

- To understand and apply the manual testing life cycle: planning, scenario and case design, execution, defect reporting and closure.
- To design effective positive and negative test cases using equivalence partitioning, boundary value analysis and error guessing.
- To execute the test cases on a real web application and record actual results truthfully.
- To write clear, reproducible defect reports with severity and priority.
- To maintain traceability between requirements, scenarios, test cases and defects.

1.9 Relevant mathematics associated with the Project

System Description:

- **Input:** Set of requirements $R = \{r_1, \ldots, r_6\}$; set of test users U; set of test cases $TC = \{tc_1, \ldots, tc_{39}\}$, each a tuple (precondition, steps, data, expected).
- **Output:** Result function $res : TC \to \{\text{Pass}, \text{Fail}\}$; defect set $D = \{d_1, \ldots, d_9\}$; traceability relation $T \subseteq R \times TC$; defect mapping $M \subseteq TC_{fail} \times D$.
- **Data structures:** tables (Excel sheets) for test cases, defects and RTM; each row is a record.
- **Functions:**
  - $design : R \to 2^{TC}$ (every requirement yields one or more test cases).
  - $execute : TC \times U \to \{\text{Pass}, \text{Fail}\}$, $res(tc) = \text{Pass} \iff actual(tc) = expected(tc)$.
  - $report : TC_{fail} \to D$ (every failed case is linked to exactly one defect; a defect may cover several cases).
- **Mathematical formulation:** Pass percentage $P = \frac{|\{tc : res(tc) = \text{Pass}\}|}{|TC|} \times 100 = \frac{28}{39} \times 100 = 71.8\%$. Defect density per module $= |D_m|/|TC_m|$. Equivalence classes for the postal-code field: valid digits, letters (invalid), empty (invalid).

- **Success Conditions:** every requirement is covered by at least one test case ($\forall r \in R : \exists tc, (r, tc) \in T$); every failed test case is linked to a defect report with evidence.
- **Failure Conditions:** site unreachable; test data not accepted; a failure without reproducible steps.

1.10 Names of Conferences / Journals where papers can be published

- IEEE International Conference on Software Testing, Verification and Validation (ICST)
- ACM International Symposium on Software Testing and Analysis (ISSTA)
- International Journal of Software Engineering and Knowledge Engineering
- SPPU-sponsored and IIT-hosted student technical conferences / workshops

1.11 Review of Conference/Journal Papers supporting Project idea

1. Myers et al. [1] define the purpose of testing as finding defects and describe black-box techniques used here.
2. Jorgensen [2] formalises equivalence-class and boundary-value testing applied to the checkout form.
3. Kaner, Falk and Nguyen [3] give practical guidance on bug reporting and severity classification.
4. Itkonen, Mäntylä and Lassenius [4, 5] show through experiments that manual, exploratory testing finds many defects that scripted testing misses.
5. Bertolini and Mota [6] compare manual testing techniques and report that systematic scripts find more defects than ad-hoc testing.
6. Garousi and Mäntylä [7] survey when to automate and when manual testing is preferable.
7. IEEE 829 [8] and ISTQB [9] define the test documentation and terminology (test plan, test case, defect) used in this project.
8. Pressman [11] supplies the project-planning and risk framework.

1.12 Plan of Project Execution

Four weeks: (1) study the application and write the test plan and scenarios, (2) design the test cases and review them with the guide, (3) execute the cases with all test users and capture evidence, (4) consolidate defects, build the RTM and summary, and write this report. See Annexure C.

CHAPTER 2
TECHNICAL KEYWORDS

2.1 Area of Project

Software Testing and Quality Assurance: manual functional testing of a web application.

2.2 Technical Keywords

1. Software and its engineering
    (a) Software testing and debugging; functional testing; acceptance testing
    (b) Software defect analysis; requirements traceability
2. Human-centered computing
    (a) Web-based interaction; usability testing
3. Information systems
    (a) Electronic commerce; online shopping

CHAPTER 3
INTRODUCTION

3.1 Project Idea

- Take a real, publicly reachable e-commerce web application and test it the way a QA engineer would: plan, design cases, execute by hand, report defects.
- Swag Labs was chosen because it contains a full purchase flow and six test users (`standard_user`, `locked_out_user`, `problem_user`, `performance_glitch_user`, `error_user`, `visual_user`) that expose different categories of defects.
- All artifacts (test plan, scenarios, Excel test cases, defect reports, RTM, screenshots) are kept in a GitHub repository so the work is reproducible.

3.2 Motivation of the Project

- Manual testing is still the first and most widely practised QA activity in industry; every tester must be able to write good test cases and bug reports.
- The LP-IV syllabus assignments (test scenarios, Excel test cases, defect reports, SRS) are naturally combined into one end-to-end project on a single application.
- Testing an e-commerce flow teaches validation of forms, calculations (totals, tax) and state handling (cart, session), which are common sources of real-world bugs.

3.3 Literature Survey

- **Myers, Sandler and Badgett (2011)** [1]: *The Art of Software Testing* states that a successful test is one that finds an error, and describes equivalence partitioning, boundary-value analysis and error guessing. These three techniques were used to design the login and checkout cases.
- **Jorgensen (2013)** [2]: formal treatment of black-box testing; the weak-normal equivalence-class approach was used for the name and postal-code fields.
- **Kaner, Falk and Nguyen (1999)** [3]: *Testing Computer Software* gives the structure of a good bug report (summary, steps, expected, actual, severity) that our defect sheet follows.
- **Itkonen, Mäntylä and Lassenius (2007)** [4]: a controlled experiment showing exploratory manual testing finds as many defects as test-case-based testing in less time; we combined both by adding error-guessing cases using the special users.
- **Itkonen, Mäntylä and Lassenius (2009)** [5]: field study of how testers use knowledge during manual testing, supporting the use of domain knowledge (e-commerce rules such as tax calculation) in expected results.
- **Bertolini and Mota (2010)** [6]: empirical comparison of manual test techniques, concluding that systematic test procedures improve defect detection.
- **Garousi and Mäntylä (2016)** [7]: systematic review of when to automate; manual testing is recommended for new features and usability, which fits this project.

- **IEEE 829-2008** [8]: standard for test documentation; our test plan sections follow it.
- **ISTQB Foundation Syllabus** [9]: definitions of test scenario, test case, severity and priority, and the defect life cycle used in the state diagram.
- **Sauce Labs documentation** [10]: describes the Swag Labs demo and its test users.

CHAPTER 4
PROBLEM DEFINITION AND SCOPE

4.1 Problem Statement

Given the Swag Labs e-commerce website, prepare and execute a complete manual test suite covering login, products, cart, checkout and navigation, and document every defect found in a structured bug report so that a developer can reproduce and fix it.

4.1.1 Goals and objectives

- Input: the application URL, the six test user accounts, the feature list (requirements) and the test data chosen by the tester.
- Output: an Excel workbook with sheets Test Cases, Defect Reports, RTM and Summary; Markdown documents; evidence screenshots.
- Objective: maximise defects found per test case while keeping every case reproducible.

4.1.2 Statement of scope

- In scope: functional, negative and UI testing of login, product listing and sorting, cart, checkout information form, order overview and completion, side menu and logout, protected-page access.
- Out of scope: payment gateway (fixed dummy card), load and security testing, browsers other than Microsoft Edge, mobile layouts.
- Input bounds: text fields on login and checkout; postal code expected numeric; six products with fixed prices.

4.2 Major Constraints

- The application is a public demo that cannot be modified; defects are reported but not fixed.
- Only the documented test users exist; new accounts cannot be created.
- The site may change without notice (a new "Dynamic Catalog" menu link appeared during testing), so expected results must be re-verified before final execution.
- Testing is manual, so each full execution of the 39 cases takes about two hours.

4.3 Methodologies of Problem solving and efficiency issues

- **Ad-hoc testing:** fast but not repeatable and gives no coverage measure.
- **Scripted manual testing (chosen):** test cases designed from requirements with black-box techniques, executed in a fixed order per module; coverage is measured through the RTM. To reduce effort, cases are grouped by test user so that each login is reused for several cases.

- **Automation:** appropriate for regression later (covered in the companion automation project) but not for first-time defect discovery.

4.4 Outcome

- 39 executed test cases (28 pass, 11 fail) in an Excel workbook.
- 9 defect reports with evidence; 1 critical, 3 high, 5 medium.
- Requirement traceability matrix and module-wise summary.

4.5 Applications

- Quality assurance of online shopping portals before release.
- A reusable test-case template and defect-report format for any web application.
- Training material for new testers (realistic defects with evidence).

4.6 Hardware Resources Required

Sr. No.	Parameter	Minimum Requirement	Justification
1	CPU Speed	1 GHz	Only a web browser and Excel are run
2	RAM	4 GB	Browser with several tabs plus spreadsheet
3	Display	1280 x 800	Needed to observe layout defects

Table 4.1: Hardware Requirements

4.7 Software Resources Required

Platform :

1. Operating System: Windows 11
2. Browser: Microsoft Edge 142
3. Documentation: Microsoft Excel / LibreOffice Calc, Markdown editor
4. Supporting scripts: Python 3 with openpyxl (to build the workbook) and Selenium (to re-capture evidence screenshots)
5. Version control: Git and GitHub

CHAPTER 5
PROJECT PLAN

5.1 Project Estimates

A waterfall-style test process was followed: plan, design, execute, report.

5.1.1 Reconciled Estimates

5.1.1.1 Cost Estimate

No software cost; all tools are free. Effort about 48 person-hours for four students (12 hours each), including two full executions of the suite.

5.1.1.2 Time Estimates

Phase	Duration
Application study, test plan, scenarios	4 days
Test case design and review	4 days
Execution with all test users, evidence capture	4 days
Defect consolidation, RTM, summary	2 days
Report and repository	4 days
Total	**18 working days**

Table 5.1: Time estimates

5.1.2 Project Resources

People: four students and one guide. Hardware: one laptop per student. Software: Edge, Excel, Python, Git/GitHub, MiKTeX.

5.2 Risk Management w.r.t. NP Hard analysis

Test-case selection for complete coverage is in general NP-hard (set cover), but with 6 requirements and 39 cases the RTM was built by hand and full coverage verified directly. Practical risks are listed below.

5.2.1 Risk Identification

Risks were identified from the scope and environment using the questionnaire in [11]: requirements (the site's features) are stable and understood; the team has the needed skills; the external dependency is the public website.

ID	Risk Description	Probability	Impact		
			Schedule	Quality	Overall
1	Demo site changes during the project	Medium	Medium	Medium	Medium
2	Site unreachable during execution	Low	Medium	Low	Low
3	Inconsistent results between testers (human error)	Medium	Low	High	Medium

Table 5.2: Risk Table

Probability	Value	Description
High	Probability of occurrence is	$> 75\%$
Medium	Probability of occurrence is	$26 - 75\%$
Low	Probability of occurrence is	$< 25\%$

Table 5.3: Risk Probability definitions [11]

Impact	Value	Description
Very high	$> 10\%$	Schedule impact or Unacceptable quality
High	$5 - 10\%$	Schedule impact or Some parts of the project have low quality
Medium	$< 5\%$	Schedule impact or Barely noticeable degradation in quality

Table 5.4: Risk Impact definitions [11]

Risk ID	1
Risk Description	Demo site changes during the project
Category	Technology / external dependency
Source	Observed: a new menu link appeared mid-project
Probability	Medium
Impact	Medium
Response	Mitigate
Strategy	Re-verify expected results and re-execute the full sheet just before submission; keep dated screenshots
Risk Status	Occurred and handled

Risk ID	2
Risk Description	Site unreachable during execution
Category	Development Environment
Source	Dependence on Internet and public hosting
Probability	Low
Impact	Medium
Response	Accept
Strategy	Schedule execution sessions with buffer time; evidence stored locally
Risk Status	Identified

Risk ID	3
Risk Description	Inconsistent results between testers
Category	Process
Source	Manual execution by four people
Probability	Medium
Impact	High
Response	Mitigate
Strategy	Precise steps and test data in every case; every Fail must have a screenshot; peer review of results
Risk Status	Mitigated

5.2.2 Risk Analysis

5.2.3 Overview of Risk Mitigation, Monitoring, Management

5.3 Project Schedule

5.3.1 Project task set

- Task 1: Study the application, list requirements, write the test plan.
- Task 2: Write test scenarios and design test cases in Excel.
- Task 3: Execute all cases with each test user; capture screenshots.
- Task 4: Consolidate failures into defect reports; build RTM and summary.
- Task 5: Repository and project report.

5.3.2 Task network

Task 1 → Task 2 → Task 3 → Task 4 → Task 5; Task 5 documentation started in parallel with Task 4.

5.3.3 Timeline Chart

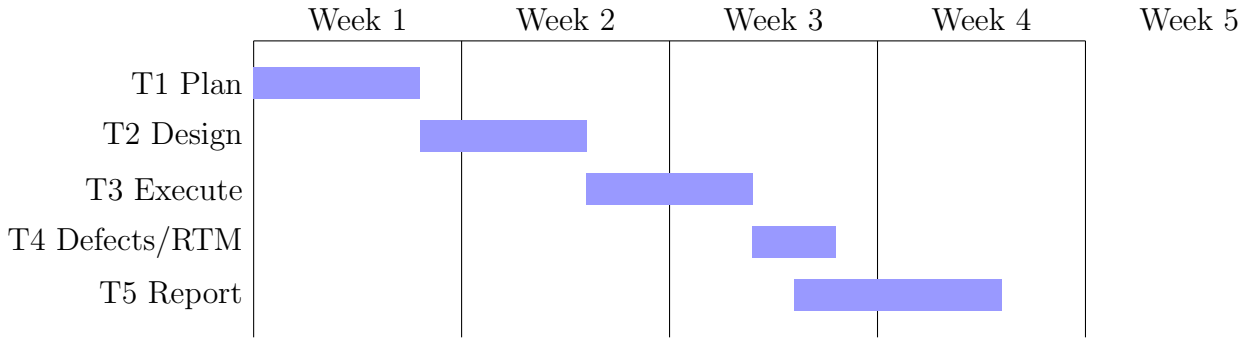


Figure 5.1: Project timeline (Gantt chart)

Please refer to Annexure C for the planner.

5.4 Team Organization

5.4.1 Team structure

Four students: one test lead (plan, RTM, review), one responsible for login and products cases, one for cart and checkout cases, one for defect reports and evidence. All members executed cases and cross-checked each other's results.

5.4.2 Management reporting and communication

Weekly review with the guide in the LP-IV slot; a shared GitHub repository holds all artifacts; a group chat for daily coordination.

CHAPTER 6

SOFTWARE REQUIREMENT SPECIFICATION

6.1 Introduction

6.1.1 Purpose and Scope of Document

This SRS summarises the functional requirements of the application under test (Swag Labs) as understood by the test team, so that test cases can be derived from them and traced back.

6.1.2 Overview of responsibilities of Developer

In this project the team acts as the QA team: derive requirements from the application, design and execute tests, and report defects. Fixing defects is the responsibility of the application's developers (Sauce Labs).

6.2 Usage Scenario

6.2.1 User profiles

- **Customer:** logs in, browses products, manages the cart and places an order.
- **Tester:** executes test cases as each of the six test users and records results.
- **Test lead:** reviews cases, maintains RTM and defect status.

6.2.2 Use-cases

Sr No.	Use Case	Description	Actors	Assumptions
1	Login	Authenticate with username and password	Customer	Test user accounts exist
2	Browse / sort products	View six products and sort by name or price	Customer	Logged in
3	Manage cart	Add, remove, view items; badge shows count	Customer	Logged in
4	Checkout	Enter name and postal code, review totals, finish order	Customer	Cart not empty
5	Logout / navigate	Use side menu, logout, reset app state	Customer	Logged in
6	Execute test case	Perform steps, compare actual vs expected, record status	Tester	Test case sheet ready
7	Report defect	Log a failure with steps, severity, priority and evidence	Tester	A test case failed

Table 6.1: Use Cases

6.2.3 Use Case View

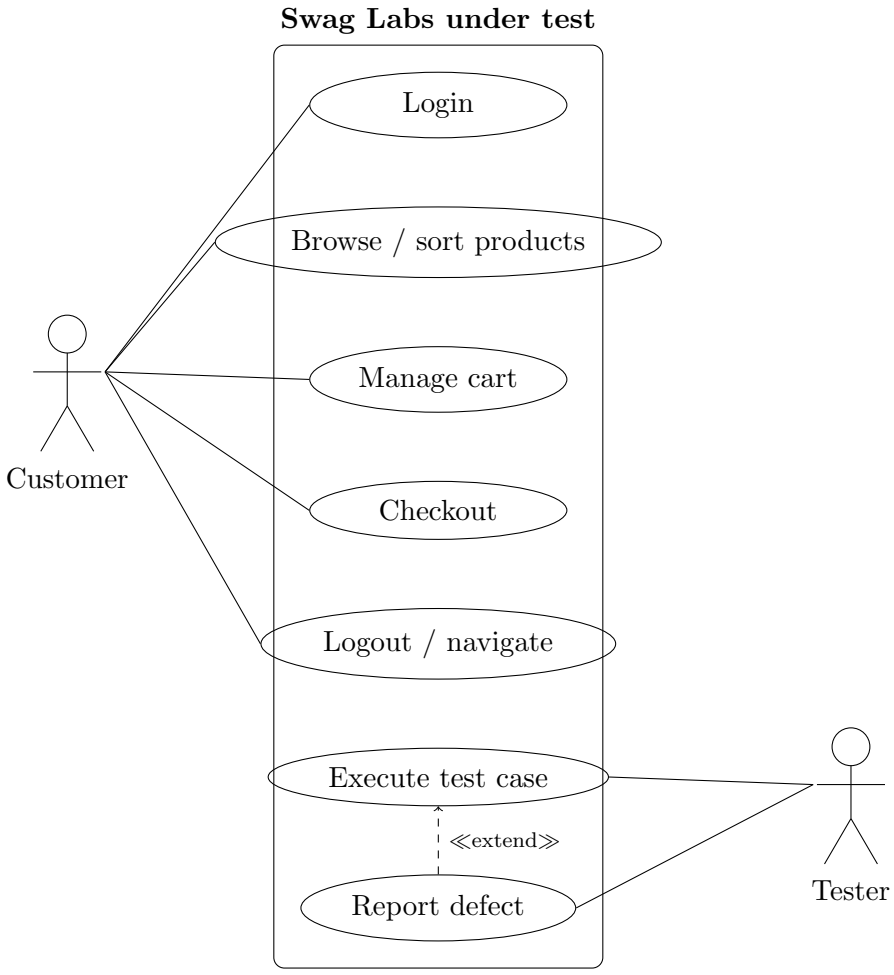


Figure 6.1: Use case diagram

6.3 Data Model and Description

6.3.1 Data Description

The testing artifacts are tabular: Test Case (ID, module, title, pre-condition, steps, test data, expected, actual, status, priority, defect ID, evidence); Defect (ID, summary, module, severity, priority, status, steps, expected, actual, test case, evidence); Requirement (ID, description, test cases).

6.3.2 Data objects and Relationships

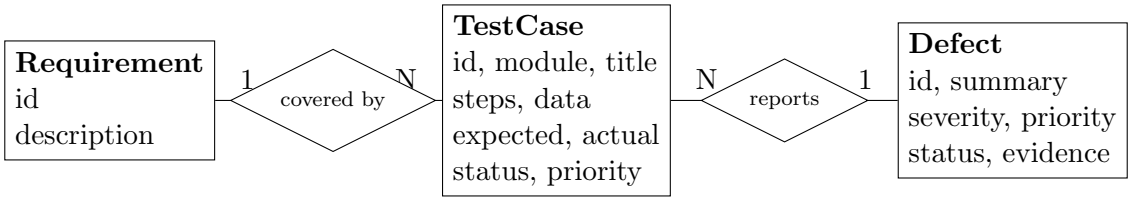


Figure 6.2: Entity relationship of test artifacts

6.4 Functional Model and Description

6.4.1 Data Flow Diagram

6.4.1.1 Level 0 Data Flow Diagram

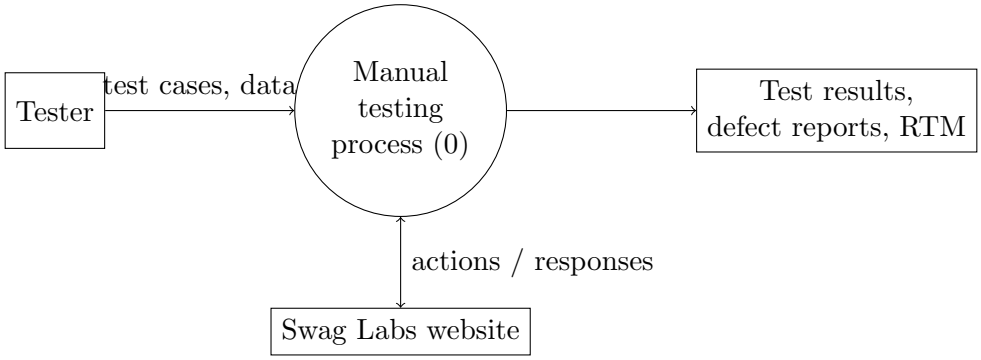


Figure 6.3: Level 0 DFD

6.4.1.2 Level 1 Data Flow Diagram

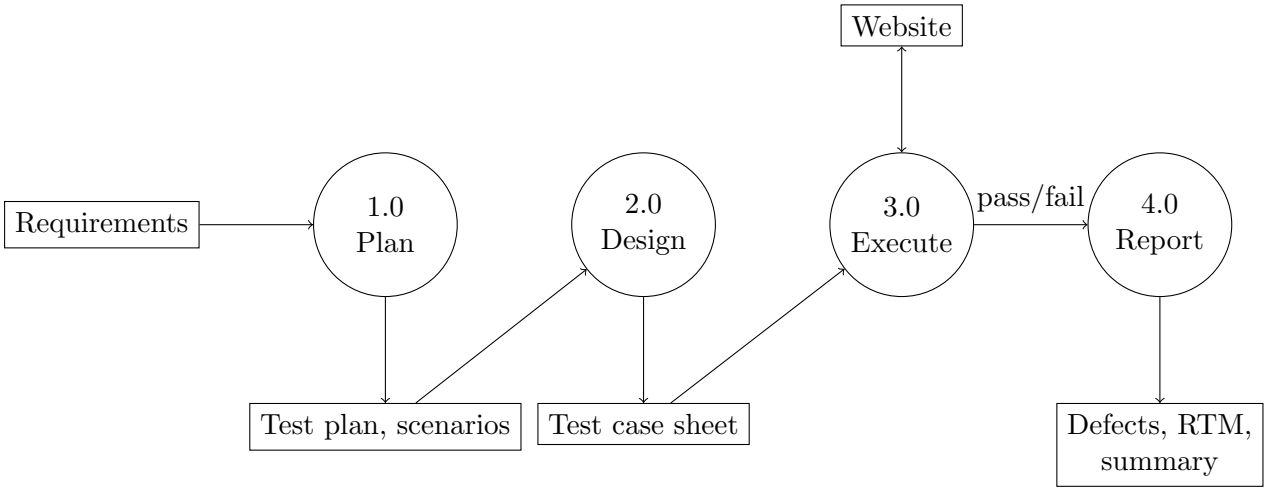


Figure 6.4: Level 1 DFD

6.4.2 Activity Diagram:

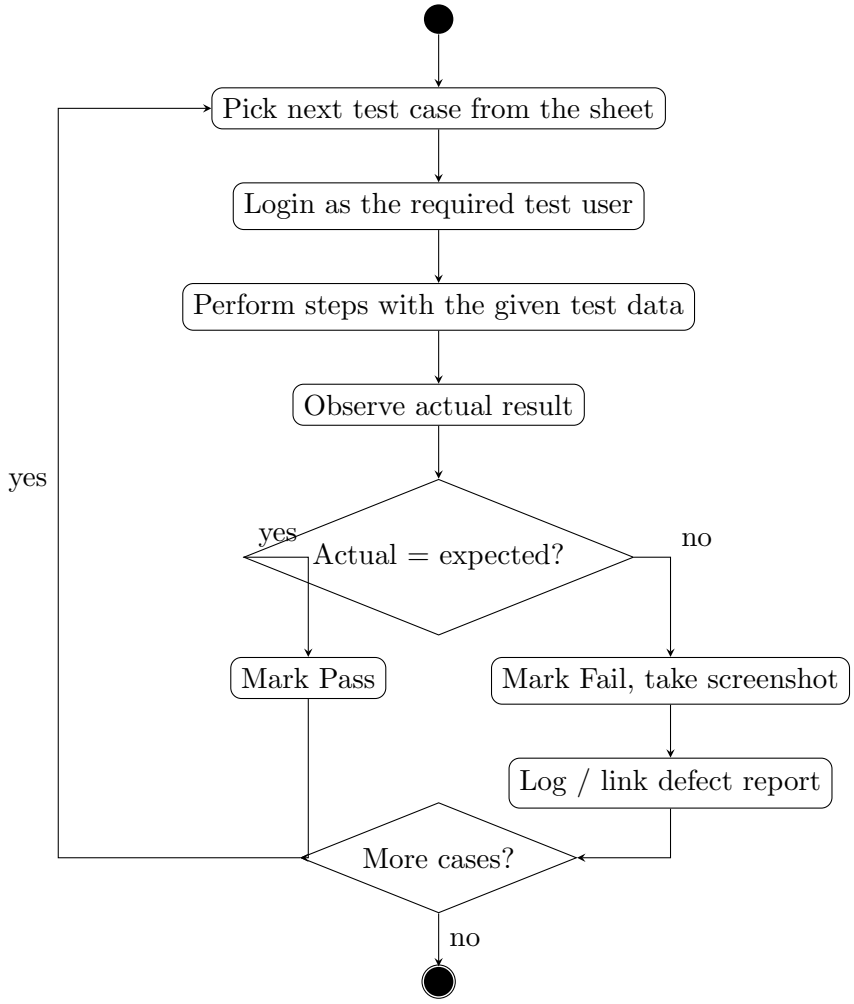


Figure 6.5: Activity diagram of test execution

6.4.3 Non Functional Requirements:

- **Interface Requirements:** the website is used through a standard desktop browser; artifacts are Excel and Markdown files readable without special tools.
- **Performance Requirements:** pages should load within 3 seconds (used as the pass criterion for the performance test case).
- **Availability:** testing depends on the public site being reachable.
- **Modifiability:** the workbook is generated from one data file so test cases can be edited and regenerated.
- **Security:** protected pages must not open without login (verified by TC_MENU_03).
- **Testability / Usability:** every case lists exact steps and data so that any tester reproduces the same result.

6.4.4 State Diagram:

Fig.6.6 shows the defect life cycle used for the Status column of the defect sheet.

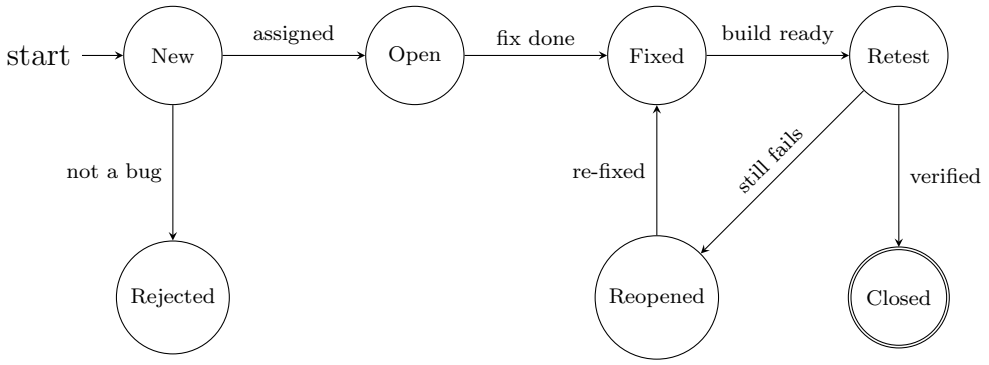


Figure 6.6: State transition diagram (defect life cycle)

6.4.5 Design Constraints

Test cases must be executable by hand without tools; every Fail must carry evidence; the workbook format must be plain Excel.

6.4.6 Software Interface Description

- **Application interface:** HTTPS web pages of Swag Labs in Microsoft Edge.
- **Artifact interface:** Excel workbook (openpyxl), Markdown files, PNG screenshots.

CHAPTER 7

DETAILED DESIGN DOCUMENT USING APPENDIX A AND B

7.1 Introduction

This chapter describes the structure of the testing process and of the artifacts produced.

7.2 Architectural Design

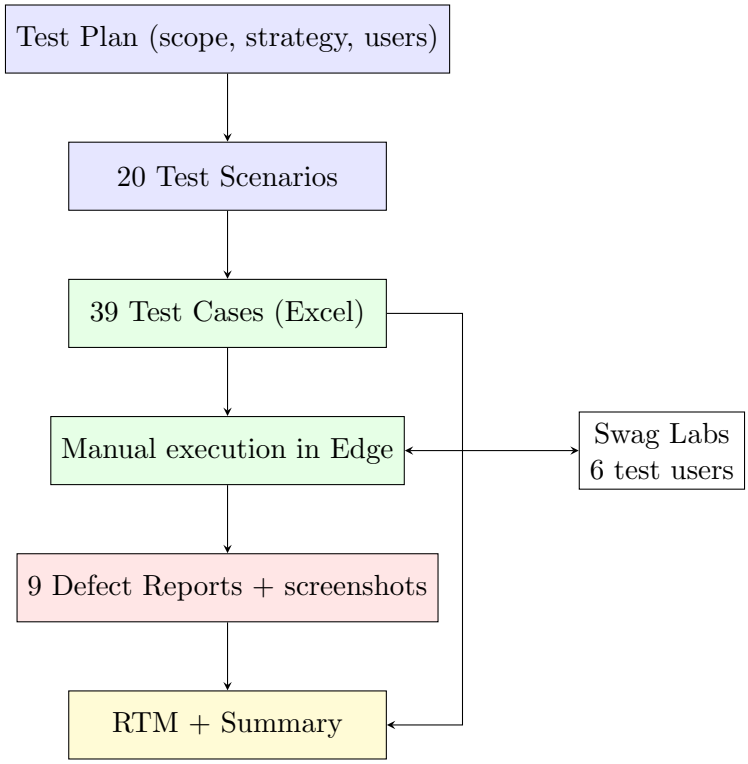


Figure 7.1: Architecture of the testing process

7.3 Data design (using Appendices A and B)

7.3.1 Internal software data structure

Each test case is a 12-field record; each defect an 11-field record; each requirement a 3-field record (see Chapter 6). The records are kept in one Python list each inside `tools/build_documents.py`.

7.3.2 Global data structure

The site URL, the common password and the severity/priority scales are global constants used by all records.

7.3.3 Temporary data structure

Counters for pass/fail per module used to build the Summary sheet.

7.3.4 Database description

No database; the Excel workbook `SwagLabs_Test_Cases.xlsx` with sheets Test Cases, Defect Reports, RTM and Summary is the persistent store.

7.4 Component Design

7.4.1 Class Diagram

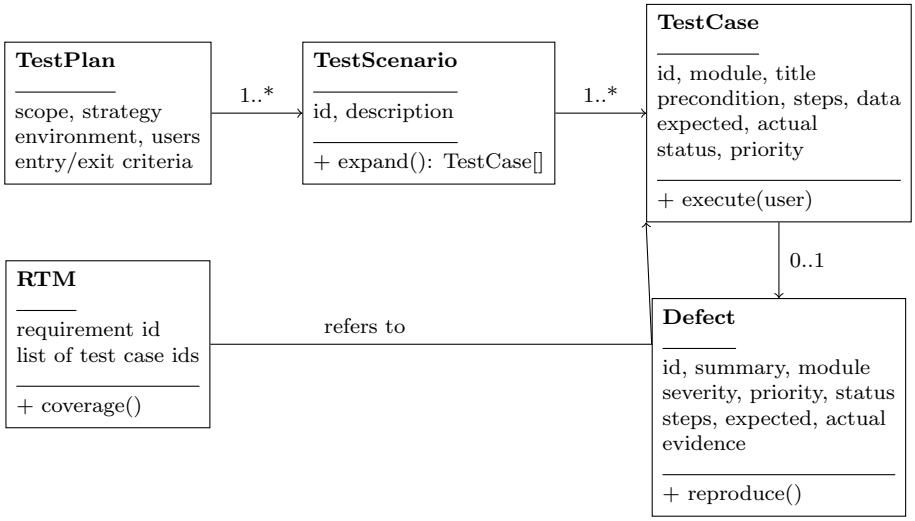


Figure 7.2: Class diagram of test artifacts

CHAPTER 8
PROJECT IMPLEMENTATION

8.1 Introduction

The project was implemented as a set of documents and an Excel workbook, supported by two small Python scripts: one that builds the workbook and Markdown files from a single data set, and one that re-captures the evidence screenshots. Repository: `https://github.com/VishwajeetTulse/ecommerce-manual-testing`.

8.2 Tools and Technologies Used

- Microsoft Edge 142 on Windows 11 for execution
- Microsoft Excel (workbook produced with Python `openpyxl`)
- Python 3.14 with `openpyxl` and `selenium` (evidence re-capture only)
- Git and GitHub; MiKTeX for this report

8.3 Methodologies/Algorithm Details

8.3.1 Algorithm 1/Pseudo Code

Algorithm 1: Test case design from a requirement

Input: requirement r, its input fields F
Output: set of test cases TC_r
add one positive case covering the normal flow of r;
foreach *field* $f \in F$ **do**
    partition the domain of f into valid and invalid classes;
    add one case per invalid class (empty, wrong type, wrong format);
    add boundary cases where f has limits;
foreach *special test user* u *relevant to* r **do**
    add an error-guessing case executed as u;
record (r, tc) in the RTM for every $tc \in TC_r$;
return TC_r;

8.3.2 Algorithm 2/Pseudo Code

Algorithm 2: Manual execution and defect logging

Input: test case sheet TC
Output: status for each case, defect list D
foreach $tc \in TC$ *grouped by test user* **do**
  log in as the required user; establish the pre-condition;
  perform the steps with the given test data; observe the actual result;
  if *actual = expected* **then**
    status ← Pass
  else
    status ← Fail; capture screenshot;
    if *an existing defect* $d \in D$ *matches* **then**
      link tc to d
    else
      create new defect with severity, priority, steps, expected, actual, evidence; add to D
compute pass percentage and module-wise summary;

Example of a designed test case (checkout postal code, invalid equivalence class):

```
TC_CHK_06  Module: Checkout   Priority: Medium
Pre-condition : cart has items; on "Checkout: Your Information"
Steps         : 1. Enter First Name "Vishwajeet"  2. Enter Last Name "Tulse"
                3. Enter Postal Code "abc"        4. Click Continue
Expected      : validation error for Postal Code
Actual        : form accepted, Overview page displayed
Status        : Fail   -> DEF-05
```

8.4 Verification and Validation for Acceptance

Acceptance criteria for the test package: every requirement has at least one test case (RTM complete); every test case has an actual result and status; every Fail is linked to a defect with a screenshot; the summary totals equal the sheet counts. All criteria were checked by the test lead; the summary sheet is generated automatically from the same data as the test-case sheet, so the counts cannot disagree.

CHAPTER 9
SOFTWARE TESTING

9.1 Type of Testing Used

- **Functional testing** of each feature (login, products, cart, checkout, navigation).
- **Negative testing** with empty, invalid and locked-out inputs.
- **UI testing** of elements, labels, alignment and button states.
- **System testing** of the end-to-end purchase flow.
- **Regression**: the full sheet was executed twice; the second run found that a new menu link had been added to the site and TC_MENU_01 was updated.

9.2 Test Cases and Test Results

Module	Test cases	Pass	Fail
Login	8	7	1
Products	8	3	5
Cart	6	4	2
Checkout	11	8	3
Menu	3	3	0
UI	3	3	0
Total	**39**	**28**	**11**

Table 9.1: Module-wise test results (pass percentage 71.8%)

TC	Title	User	Status
TC_LOGIN_03	Login with valid credentials	standard_user	Pass
TC_LOGIN_04	Login with empty username and password	–	Pass
TC_LOGIN_06	Login with locked-out user	locked_out_user	Pass
TC_LOGIN_08	Login response time	performance_glitch_user	Fail
TC_PROD_02	Sort products by price low to high	standard_user	Pass
TC_PROD_04	Product images match the product	problem_user	Fail
TC_PROD_05	Sort by price works	problem_user	Fail
TC_PROD_06	Sort dropdown works	error_user	Fail
TC_PROD_07	Prices match the catalogue	visual_user	Fail
TC_CART_01	Add two products and verify badge	standard_user	Pass
TC_CART_04	Add to cart works for all products	problem_user	Fail
TC_CART_05	Cart is cleared on logout	standard_user	Fail
TC_CHK_01	Checkout with empty information form	standard_user	Pass
TC_CHK_05	First name rejects numeric input	standard_user	Fail
TC_CHK_06	Postal code rejects alphabetic input	standard_user	Fail
TC_CHK_07	Overview totals, tax 8% and total	standard_user	Pass
TC_CHK_09	Finish order shows confirmation	standard_user	Pass
TC_CHK_10	Last Name field accepts input	problem_user	Fail
TC_MENU_03	Protected page cannot be opened without login	–	Pass

Table 9.2: Selected test cases (full list of 39 in the Excel workbook)

ID	Summary	Severity	Priority
DEF-01	Wrong product images for problem_user	High	Medium
DEF-02	Price sorting does not reorder products for problem_user	Medium	Medium
DEF-03	Add to cart has no effect for some products (problem_user)	High	High
DEF-04	Last Name field cannot be filled during checkout (problem_user)	Critical	High
DEF-05	Checkout form accepts invalid data (all users)	Medium	Medium
DEF-06	Cart contents persist after logout and re-login	Medium	Low
DEF-07	Sorting shows a JavaScript alert for error_user	Medium	Medium
DEF-08	Login takes about 7 seconds for performance_glitch_user	Medium	Low
DEF-09	Wrong prices and misaligned layout for visual_user	High	Medium

Table 9.3: Defects logged

CHAPTER 10

RESULTS

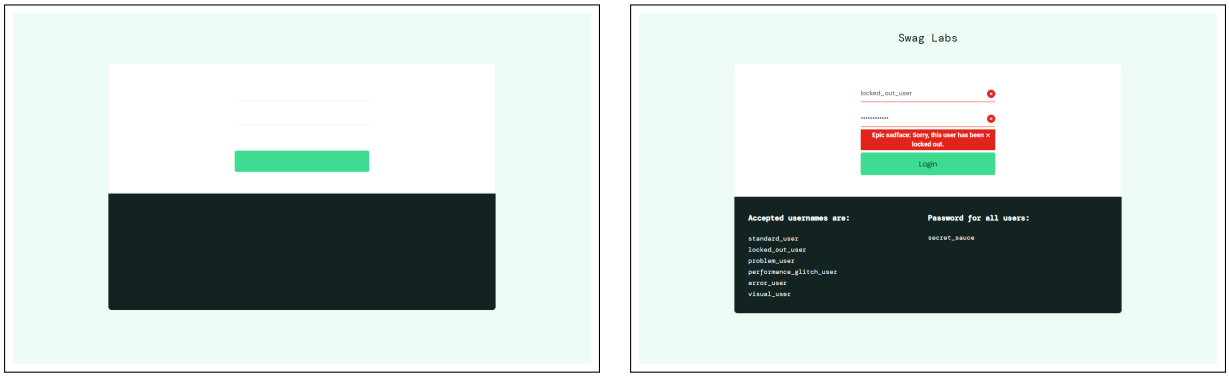
10.1 Screen shots

ID	Module	Title	Test Data	Expected Result	Actual Result	Status	Defect
TC_LOGIN_01	Login	Verify login page elements are displayed	-	Username field, Password field, Login button and Swag Labs logo are visible; accepted usernames are listed	All elements displayed as expected	Pass	
TC_LOGIN_02	Login	Verify placeholders of the input fields	-	Placeholders 'Username' and 'Password' are shown	Placeholders shown correctly	Pass	
TC_LOGIN_03	Login	Login with valid credentials	standard_user / secret_sauce	User is redirected to Products page (/inventory.html)	Products page displayed	Pass	
TC_LOGIN_04	Login	Login with empty username and password	blank / blank	Error 'Epic sadface: Username is required' is shown	Error message shown as expected	Pass	
TC_LOGIN_05	Login	Login with valid username and wrong password	standard_user / wrong	Error 'Username and password do not match any user in this service'	Error message shown as expected	Pass	
TC_LOGIN_06	Login	Login with locked-out user	locked_out_user / secret_sauce	Error 'Sorry, this user has been locked out.' is shown	Error message shown as expected	Pass	
TC_LOGIN_07	Login	Password field masks the typed characters	secret_sauce	Characters are shown as dots	Characters masked	Pass	
TC_LOGIN_08	Login	Login response time for performance_glitch_user	performance_glitch_user / secret_sauce	Products page loads within 3 seconds	Products page took about 7 seconds to load	Fail	DEF-08
TC_PROD_01	Products	Verify six products are listed with name, image, description and price	-	6 products displayed, each with a unique image, name, description and price	6 products displayed correctly	Pass	
TC_PROD_02	Products	Sort products by price low to high	lohi	Products ordered 7.99, 9.99, 15.99, 15.99, 29.99, 49.99	Order 7.99, 9.99, 15.99, 15.99, 29.99, 49.99	Pass	
TC_PROD_03	Products	Sort products by name Z to A	za	Products listed in reverse alphabetical order	Reverse alphabetical order shown	Pass	
TC_PROD_04	Products	Product images match the product (problem_user)	problem_user / secret_sauce	Each product shows its own image	Every product shows the same dog image	Fail	DEF-01

Figure 10.1: Test case sheet (first rows of the Excel workbook)

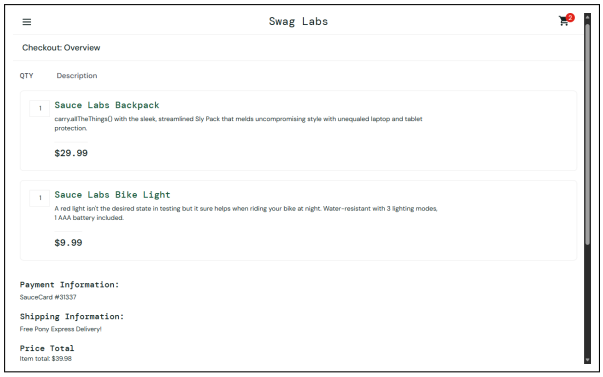
ID	Summary	Module	Severity	Priority	Status	Expected	Actual	Test Case
DEF-01	Wrong product images for problem_user	Products	High	Medium	Open	Each product shows its own image	All six products show the same dog image	TC_PROD_04
DEF-02	Price sorting does not reorder products for problem_user	Products	Medium	Medium	Open	Products reordered 7.99 ... 49.99	Order stays 29.99, 9.99, 15.99, 49.99, 7.99, 15.99	TC_PROD_05
DEF-03	Add to cart has no effect for some products (problem_user)	Cart	High	High	Open	Badge shows 2 and buttons change to Remove	Nothing happens; no badge, buttons unchanged	TC_CART_04
DEF-04	Last Name field cannot be filled during checkout (problem_user)	Checkout	Critical	High	Open	Overview page displayed	Text typed into Last Name replaces First Name; Last Name stays empty; 'Error: Last Name is required'. Checkout is blocked.	TC_CHK_10
DEF-05	Checkout form accepts invalid data (all users)	Checkout	Medium	Medium	Open	Validation errors for name and postal code	Form accepted; Overview page displayed	TC_CHK_05, TC_CHK_06
DEF-06	Cart contents persist after logout and re-login	Cart	Medium	Low	Open	Cart is empty after fresh login	Badge shows 1; Onesie still in cart	TC_CART_05
DEF-07	Sorting shows a JavaScript alert for error_user	Products	Medium	Medium	Open	Products reordered	Alert 'Sorting is broken! This error has been reported to Backtrace.'	TC_PROD_06
DEF-08	Login takes about 7 seconds for performance_glitch_user	Login	Medium	Low	Open	Page loads within 3 seconds	About 6.8 seconds	TC_LOGIN_08
DEF-09	Wrong prices and misaligned layout for visual_user	Products	High	Medium	Open	Backpack $29.99, Bike Light $9.99; cart icon aligned in header	Backpack $69.96, Bike Light $91.76, Bolt T-Shirt $14.41, Fleece Jacket $47.3; cart icon drops below header; titles shifted	TC_PROD_07, TC_PROD_08

Figure 10.2: Defect report sheet

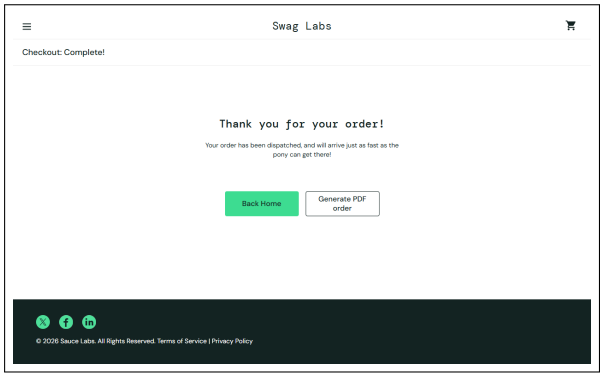


(a) Login page (TC_LOGIN_01) (b) Locked-out user error (TC_LOGIN_06)

Figure 10.3: Login test evidence

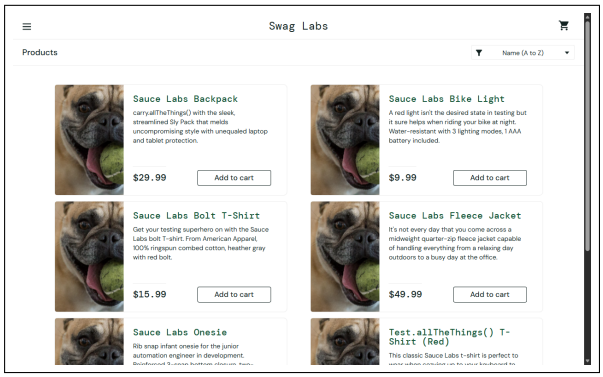


(a) Checkout overview, totals correct (TC_CHK_07)

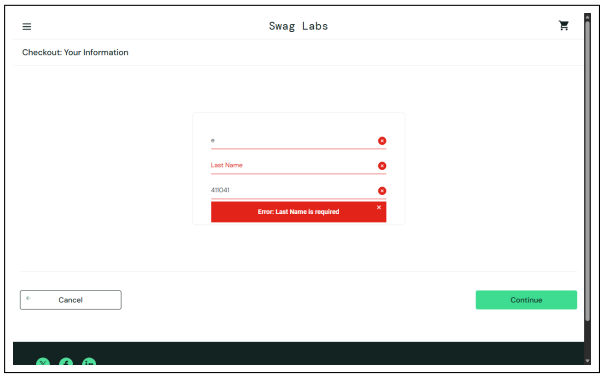


(b) Order complete (TC_CHK_09)

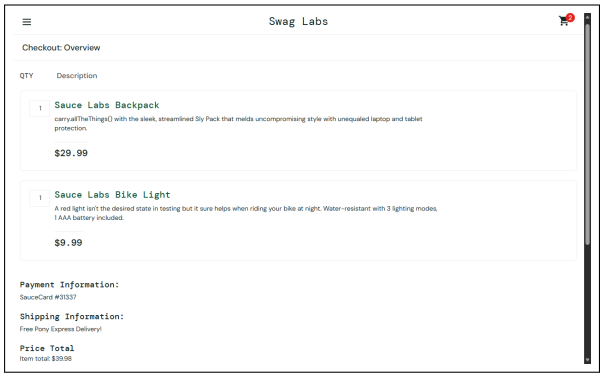
Figure 10.4: Checkout happy-path evidence



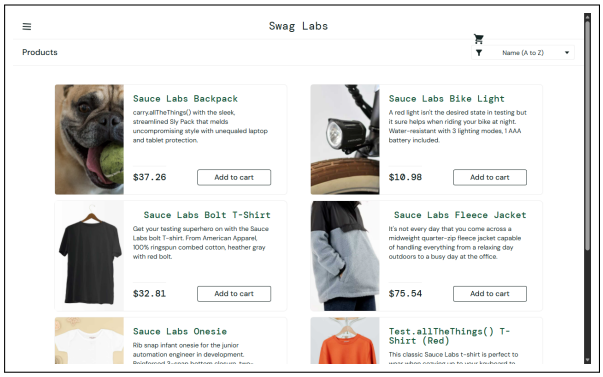
(a) DEF-01 wrong product images



(b) DEF-04 Last Name cannot be filled



(c) DEF-05 invalid data accepted



(d) DEF-09 wrong prices, misaligned cart icon

Figure 10.5: Defect evidence

10.2 Outputs

- `test_cases/SwagLabs_Test_Cases.xlsx`: 39 test cases, 9 defects, RTM (6 requirements), summary.
- `docs/Test_Plan.md`, `docs/Test_Scenarios.md`, `docs/Test_Cases.md`, `docs/Defect_Reports.md`
- `screenshots/`: 23 evidence images.

Summary sheet values: total 39, passed 28, failed 11, pass percentage 71.8%, defects 9 (critical 1, high 3, medium 5, low 0).

CHAPTER 11
DEPLOYMENT AND MAINTENANCE

11.1 Installation and un-installation

No installation is required to use the test package: open the Excel workbook and the Markdown files. To regenerate them:

```
git clone https://github.com/VishwajeetTulse/ecommerce-manual-testing
pip install openpyxl selenium
python tools/build_documents.py      # rebuild workbook and Markdown
python tools/capture_evidence.py     # re-capture screenshots (Edge/Chrome)
```

11.2 User help

To execute the suite: open `https://www.saucedemo.com`, take the Test Cases sheet, filter by Module, log in as the user named in Test Data (password `secret_sauce`), follow the steps, compare with Expected Result, fill Actual Result and Status, and for every Fail add a screenshot and a row in the Defect Reports sheet. Use the side-menu "Reset App State" between cases that need an empty cart.

CHAPTER 12
CONCLUSION AND FUTURE SCOPE

This mini project carried out the complete manual testing process on a real e-commerce website. Starting from a test plan and twenty scenarios, thirty-nine test cases were designed with black-box techniques and executed by hand with six test users. Eleven failures were traced to nine defects, documented with severity, priority, reproduction steps and screenshots, and a traceability matrix links every requirement to its tests. The work shows that systematic manual testing with well-chosen negative cases and special test users finds a wide range of functional, visual and performance defects.

Future scope: extend coverage to accessibility and cross-browser testing, add exploratory testing sessions with charters, convert the stable passing cases into automated regression scripts (done in the companion automation project), and track defects in a tool such as Jira with the life cycle shown in Fig.6.6.

CHAPTER 13
REFERENCES

[1] G. J. Myers, C. Sandler and T. Badgett. 2011. *The Art of Software Testing* (3rd ed.). John Wiley & Sons.

[2] P. C. Jorgensen. 2013. *Software Testing: A Craftsman's Approach* (4th ed.). CRC Press.

[3] C. Kaner, J. Falk and H. Q. Nguyen. 1999. *Testing Computer Software* (2nd ed.). John Wiley & Sons.

[4] J. Itkonen, M. V. Mäntylä and C. Lassenius. 2007. Defect detection efficiency: test case based vs. exploratory testing. In *Proc. First International Symposium on Empirical Software Engineering and Measurement (ESEM)*. IEEE, 61–70.

[5] J. Itkonen, M. V. Mäntylä and C. Lassenius. 2009. How do testers do it? An exploratory study on manual testing practices. In *Proc. ESEM 2009*. IEEE, 494–497.

[6] C. Bertolini and A. Mota. 2010. A framework for GUI testing based on use case design. In *Proc. ICST Workshops*. IEEE, 252–259.

[7] V. Garousi and M. V. Mäntylä. 2016. When and what to automate in software testing? A multi-vocal literature review. *Information and Software Technology* 76 (2016), 92–117.

[8] IEEE. 2008. *IEEE Standard for Software and System Test Documentation* (IEEE Std 829-2008).

[9] ISTQB. 2018. *Certified Tester Foundation Level Syllabus*. International Software Testing Qualifications Board.

[10] Sauce Labs. *Swag Labs sample application*. `https://www.saucedemo.com`

[11] R. S. Pressman. 2010. *Software Engineering: A Practitioner's Approach* (7th ed.). McGraw-Hill.

ANNEXURE A

LABORATORY ASSIGNMENTS ON PROJECT ANALYSIS OF ALGORITHMIC DESIGN

- **IDEA Matrix.** Increase: defect detection. Drive: requirement coverage. Educate: testers on bug reporting. Accelerate: regression through reusable cases. Improve: input validation of the site. Deliver: Excel test package. Evaluate: pass percentage per module. Associate: failures with defects. Ignore: payment internals. Decrease: ambiguity in steps. Eliminate: unreproducible failures. Avoid: untraced cases.
- **Feasibility.** Minimal test-suite selection is NP-hard in general (set cover), but with 6 requirements and 39 cases the RTM was completed by inspection. Execution cost is linear in the number of cases.
- **Function.** $x = (\text{test case, user})$, $y = f(x) \in \{\text{Pass}, \text{Fail}\}$ with $f(x) = \text{Pass} \iff actual = expected$.

ANNEXURE B

LABORATORY ASSIGNMENTS ON PROJECT QUALITY AND RELIABILITY TESTING OF PROJECT DESIGN

- **Objects and morphisms.** Objects: Requirement, Scenario, TestCase, Defect. Morphisms: expand (Scenario → TestCase), execute (TestCase → status), report (failed TestCase → Defect), trace (Requirement → TestCase).
- **Functional dependency graph.** Defect depends on TestCase; TestCase depends on Scenario and Requirement; Summary depends on all TestCase statuses (see Fig.6.4).
- **Test data.** Equivalence classes: username ∈ {valid, empty, unknown, locked}; password ∈ {correct, wrong, empty}; first/last name ∈ {letters, digits, symbols, empty}; postal code ∈ {digits, letters, empty}.
- **Black-box test cases.** 39 cases in the workbook; selection in Table 9.2.

ANNEXURE C
PROJECT PLANNER

Week	Activity	Deliverable	Status
1	Study Swag Labs, list requirements, write test plan and scenarios	Test_Plan.md, Test_Scenarios.md	Done
2	Design 39 test cases in Excel; guide review	SwagLabs_Test_Cases.xlsx	Done
3	Execute with six users, capture evidence, log defects	Status column, screenshots, Defect sheet	Done
4	RTM, summary, repository, report	RTM sheet, report.pdf	Done

Table C.1: Project planner

ANNEXURE D

REVIEWERS COMMENTS OF PAPER SUBMITTED

Not applicable for this mini project.

ANNEXURE E
PLAGIARISM REPORT

To be attached after checking through the institute's plagiarism tool.

ANNEXURE F

TERM-II PROJECT LABORATORY ASSIGNMENTS

1. Review of design: the guide's suggestion to use all six test users, not only `standard_user`, was adopted and produced most of the defects.

2. Project workstation: Windows 11 laptop with Microsoft Edge 142, Excel, Python 3.14 and Git.

3. Execution: completed as described in Chapters 8 and 9.

4. Test results: 39 cases, 28 pass, 11 fail, 9 defects; module-wise chart in Table 9.1.

ANNEXURE G

INFORMATION OF PROJECT GROUP MEMBERS

1. Name : Vishwajeet Tulse
2. Date of Birth :
3. Gender :
4. Permanent Address :
5. E-Mail :
6. Mobile/Contact No. :
7. Placement Details :
8. Paper Published : –

1. Name : Student Name 2
2. Date of Birth :
3. Gender :
4. Permanent Address :
5. E-Mail :
6. Mobile/Contact No. :
7. Placement Details :
8. Paper Published : –

1. Name : Student Name 3
2. Date of Birth :
3. Gender :
4. Permanent Address :
5. E-Mail :
6. Mobile/Contact No. :
7. Placement Details :
8. Paper Published : –

1. Name : Student Name 4
2. Date of Birth :
3. Gender :
4. Permanent Address :
5. E-Mail :
6. Mobile/Contact No. :
7. Placement Details :
8. Paper Published : –